APK Audit
プログラムの仕組みと
実装解説

開発意図を、APKの静的解析と
Androidの動的観測で確認するまで

対象	Pythonパッケージ `apk-intent-audit` 0.1.0、意図1.1拡張、研究用補助ランナー
確認日	2026年11月9日（実験結果の整理は11月6日版を参照）
資料の役割	発表スライドとは独立した、実装とデータ処理の技術解説
読み方	第1〜3節で全体像、第4〜12節で処理、第13〜16節で実例と運用を確認

入力は「APK・事前に宣言した意図・操作シナリオ」。
出力は「意図との照合結果・静的と動的の対応・原本への参照」。

この資料は、現時点のソースコードを読んで処理を説明する。コード中の条件と実験で確認した範囲を分け、未実行の機能を実証済みとは扱わない。新しいAndroid実験は実施していない。

目次

1 最初に理解すること 2

2 システム全体の構成 3

3 入力データの契約と検証 4

4 統一パイプラインの制御 6

5 静的解析の詳しい仕組み 7

6 補助機能：URL引数の局所的な文脈解析 9

7 動的取得：Androidをどのように動かすか 10

8 原本を共通の通信フローへ変換する 12

9 時計補正と操作区間の対応 15

10 開発意図との照合器 17

11 静的・動的の対応表と日本語説明 20

12 証拠の保存とレポート生成 21

13 実際の1試行を処理順に読む 23

14 研究用ランナーの仕組みと実験の位置付け 24

15 実際に使うときの読み方と操作 25

16 実装の境界と改善する場合の場所 27

A ソースコードへの案内 28

B 実装抜粋：帰属と入力区間 29

C 用語と実装資料 30

1 最初に理解すること

1.1 何のためのプログラムか

自作アプリやAI支援で作成したアプリに、開発者が把握していない通信先・権限・SDK候補が含まれていないかを確認するための研究用ツールである。開発者は解析前に「接続してよい先」「使用してよい権限」などをJSONで宣言する。プログラムは、その宣言をAPK内の情報および実行中に得た通信記録と照合し、根拠付きで結果を表示する。

たとえば「利用者が指定したフィードURLだけへ接続する」と宣言したアプリが、指定したURLとは別の固定先へHTTP要求を出した場合、観測記録を根拠として不一致候補を提示する。未観測の場合には、観測時間や取得状態を残して判断を保留する。

1.2 静的解析と動的解析の違い

観点	静的解析	動的観測
調べるもの	APKのManifest、DEX、リソース	操作中のパケット、UID、操作時刻
実行の要否	APKを起動せず調べる	Emulatorで対象APKを実行する
主な情報	権限宣言、URL文字列、SDK特徴	宛先、方式、接続試行、HTTP要求等
記録の意味	宣言・実行され得る候補	今回の取得範囲に存在する観測
典型的な不足	実行時の入力や生成値を解決できない	操作していない機能や窓外の通信が見えない

静的な「候補」と動的な「観測」は、違う種類の証拠である。

URL文字列がAPKに含まれるだけでは、通信したとはいえない。通信が見えた場合も、その情報だけではコード上の送信箇所、送信目的、暗号化された本文まで確定しない。本ツールはこの差をデータ構造の段階から分けて保持する。

1.3 プログラムが自動で行う範囲

APKの検査、静的ワーカー起動、選択したEmulatorへのインストール、収集器の開始・停止、登録シナリオの実行、回収、照合、HTML生成を行う。利用者は初期環境を用意し、必要なAndroidの制御・VPN同意を確認する。任意のアプリの全画面を自動探索する方式ではなく、調べたい機能のシナリオを指定する方式である。

1.4 説明文はどのように作るか

通常の解析処理は、規則と定型文による決定的な処理である。LLMへAPKや通信ログを送って判定させる呼び出しは、この実行経路にない。開発・再検証・文章作成に使ったAI支援と、配布プログラムの実行時の仕組みを区別する。

実装：`pipeline.py/compare.py/explanation.py`、`analyze/compare/explain`。

2 システム全体の構成

2.1 実行場所とコンポーネント

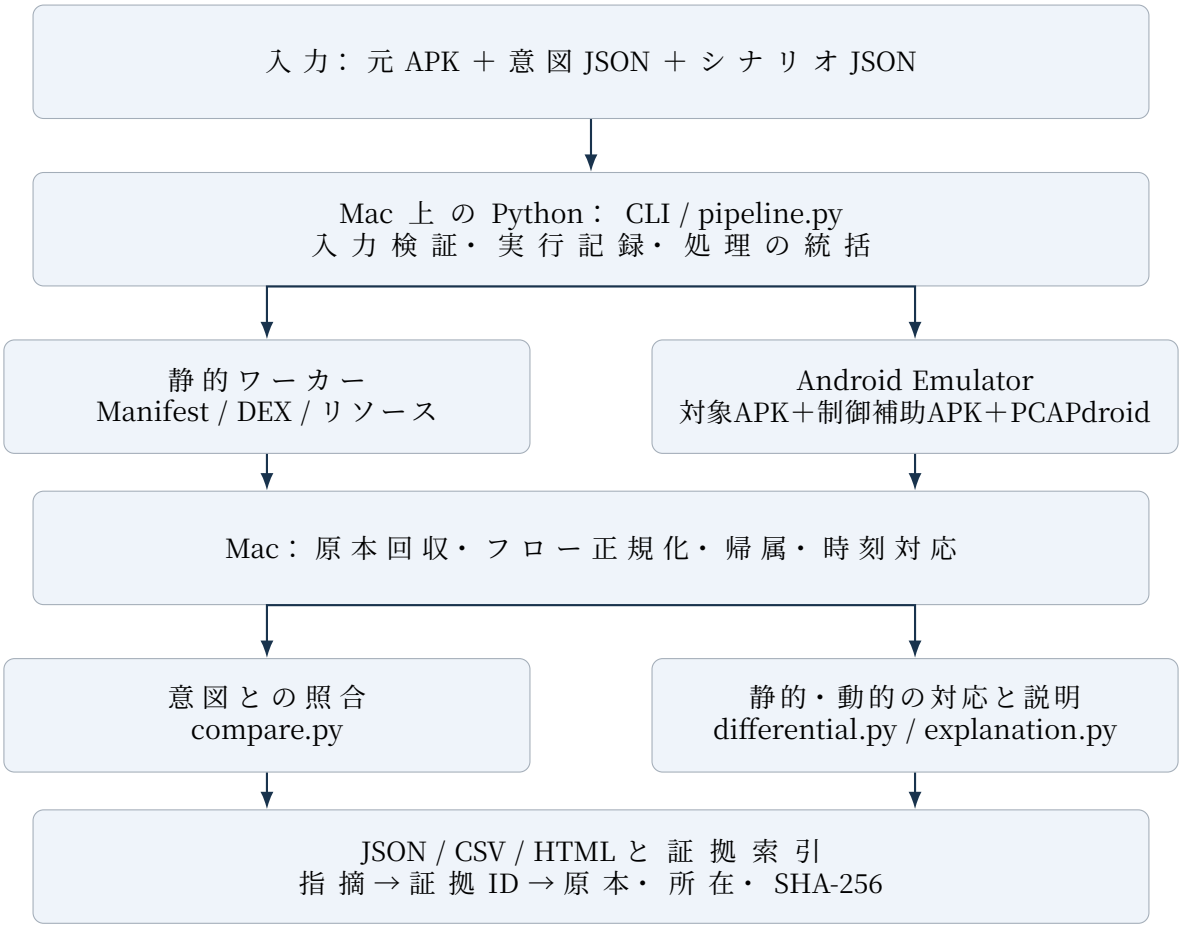


図 1: 実装上の役割分担。処理は同時実行を必須とせず、通常は静的解析の後に動的取得を行う。

2.2 自作部分と既存ツール

構成要素	担当
Pythonの自作部分	実行統括、入力検証、帰属・時刻処理、意図照合、差分、証拠索引、表示
Androguard 4.1.3	APK、バイナリManifest、DEXなどの解析基盤
ADB / Android SDK	対象端末へのコマンド実行、インストール、回収
PCAPdroid 2.0.2	VPN方式の取得、UID拡張付きPCAPの出力
自作Java補助APK	公式CaptureCtrlの要求と結果の中継、前後対照HTTP要求
dpkt 1.9.8	パケット、HTTP、TLS構造の読取り
jsonschema / defusedxml	JSON契約検証 / XML読取りの一部

実装：`pyproject.toml/automatic.py`、`dependencies/execute`。

3 入力データの契約と検証

3.1 3種類の主要入力

APKは改変しない単体APKを対象とする。split APKやAABの一括インストールは、この自動経路では扱わない。意図JSONは比較条件、シナリオJSONは実行する操作を表す。この2つを分けることで、「何を許すか」と「今回何を試したか」を区別する。

入力	必須・既定	保存方法
APK	`--apk`で指定	hashを記録し、`raw/original.apk`へ複製
意図	統一CLIでは省略可能、意図拡張ランナーでは必須	`intent.json`、1.1では宣言時点の記録も保存
シナリオ	省略時は起動し30秒観測	`scenario.json`と`run.json`内の登録内容
既存取得	再解析時にcaptureとmetadataを組で指定	APK hashを照合し、取得元を分離して保存

3.2 意図JSONの実例

以下は既存の例と同じ構造の有効なJSONである。ネットワークを「登録した利用者入力先、または明示した許可先」に限定し、権限はINTERNETだけを許可する。

コード 1: 意図1.1の実例。説明文ではなく構造化フィールドを評価する。

```
{
  "schema_version": "1.1",
  "intent_id": "feed-reader-user-input-v1",
  "app_purpose": "Fetch only recorded input feeds.",
  "network": {
    "mode": "allowlist_or_recorded_inputs",
    "allowed_destinations": [],
    "recorded_input_protocols": ["http", "https"]
  },
  "permissions": {"mode": "allowlist", "allowed": ["android.permission.INTERNET"]},
  "sdk": {"mode": "deny_all"},
  "cleartext_policy": "allow",
  "background_network_policy": "forbid"
}
```

`app_purpose`は人が読む目的の説明であり、この自然言語文から禁止先や許可先を推定する処理はない。判定する条件は`network`等の構造化フィールドに記述する。空の`allowed_destinations`を使えるのは、この例では登録入力先を許可する1.1拡張モードだからである。通常の`allowlist`モードには、1件以上の許可項目が必要になる。

実装：`contracts.py/intent_extensions.py`、`validate_intent/extended_schema`。

3.3 意図のモードと未指定の意味

モード	評価上の意味
unspecified	条件が与えられていない。比較対象が十分でも通常は要確認
deny_all	明示的に禁止。対象の接続試行や宣言を規則ごとに比較
allowlist	明示したホスト・ポート等、権限名、SDK IDの集合へ照合
allowlist_or_recorded_inputs	固定許可先に加え、証拠条件を満たす登録HTTP(S)入力先を扱う

ホストは小文字化、末尾ドット除去、IDNA変換、IP表記の正規化を行う。サブドメイン許可はinclude_subdomainsで明示し、文字列の単純な部分一致で許可範囲を広げない。

3.4 シナリオJSON

コード 2: 説明用の操作例。URIとActivityは対象アプリに合わせて指定する。

```
{
  "schema_version": "1.0",
  "scenario_id": "mechanism-explanation-v1",
  "execution": "automatic",
  "repetitions": 1,
  "steps": [
    {
      "id": "launch",
      "action": "launch",
      "observe_s": 5
    },
    {
      "id": "input",
      "action": "open_uri",
      "activity": "org.example.apkaudit.compare.input/.MainActivity",
      "uri": "http://10.0.2.2:18762/feed/example.xml",
      "observe_s": 30
    }
  ]
}
```

analyzeで受け付けるのは自動実行1反復である。反復は出力先を分けて行う。ステップIDは一意でなければならず、manual_checkpointや未処理の事前条件リストはこの経路では受け付けない。

open_uriは対象パッケージ内のActivityを明示し、HTTP(S) URLを渡す。資格情報・フラグメント・制御文字等を含むURIを拒否する。tapでは押すresource IDと、操作後に確認するresource IDの両方を要求する。

3.5 解析前に意図を記録する理由

意図1.1では、静的解析を呼ぶ前に宣言内容と読み込みUTC時刻をメモリ上で確保し、その後raw/intent-declaration.jsonへ書く。照合時はhash付き索引、宣言内容、読み込み時点と実行モードを確認し、通常の事前記録と既存ログの事後再解析を別のラベルにする。これはローカルの記録であり、独立した時刻認証サービスではない。

実装：pipeline.py/contracts.py/compare.py、scenario_input/validate_open_uri/compare。

4 統一パイプラインの制御

4.1 入口から出力までの呼出し順

1. `cli.main()`が引数を解析し、`pipeline.analyze()`へ渡す。
2. 出力先が空か、モードの組合せが正しいか、意図・シナリオが契約に合うかを確認する。
3. `static_stage()`がrunを作り、静的ワーカーを起動する。
4. 意図、シナリオ、実行コードのhash一覧を保存する。
5. liveでは`automatic.execute()`、replayでは`import_capture()`を呼ぶ。staticでは動的処理を省略する。
6. `compare()`で指摘JSONを作り、`report()`で詳細HTMLとCSVを生成する。
7. `differential.build()`で対応表を作り、`explain()`と`summary.html`を生成する。

4.2 状態は1個の成功フラグではない

`run.json`の項目	表すもの
`analysis_mode`	live / replay / staticという実行形態
`static_status`	静的解析がpending / completed / partial / failed等のどこにあるか
`execution_status`	Android上の操作実行が完了したか
`dynamic_status`	動的取得を行ったか、原本を取り込めたか、その品質
`run_status`	段階別状態を踏まえた全体の状態
`errors`	失敗段階、例外名、メッセージ

動的処理が`partial`でも、対象のHTTP要求が取得され、照合可能なことはある。逆に、実行完了やHTML生成だけでは取得範囲の完全性を意味しない。後述の対照試験に合格したという判定も、取得の完全性とは別の確認である。

4.3 失敗時にも残すもの

run作成後の静的失敗は`run.json`に理由を残し、統一入口は保存できた範囲からレポートを生成する。動的処理の例外は原本を保持し、停止処理を試みる。出力先が空でない場合は開始前に拒否し、過去の証拠を上書きしない。

ただし、入力検証でrun作成前に停止する場合や、証拠整合性の検査自体が失敗する場合は、通常レポートまで到達しないことがある。この場合はCLIのエラーと保存済みファイルを確認する。

実装：`pipeline.py/cli.py/core.py`、`analyze/static_stage/new_run`。

5 静的解析の詳しい仕組み

5.1 APKの事前検査とワーカー分離

APKはZIP形式として検査する。既定上限は200 MiB、展開後サイズの合計は1 GiB、メンバー数は10万である。絶対パス、親参照、重複名、暗号化ZIP項目などを拒否し、ManifestとDEXの存在を確認する。ここでの検査は入力の構造を制限するものであり、APKの安全性判定ではない。

静的解析は別のPythonプロセスで動かす。親側で実時間タイムアウトを設け、`raw/static-worker.log`と`static-worker.json`へ診断情報を残す。既定タイムアウトは600秒である。

Linux等ではアドレス空間上限を3 GiBまたは継承上限の小さい方へ設定する。Darwinでは`RLIMIT_AS`を適用せず、その理由を結果へ明記する。Macで発生した「current limit exceeds maximum limit」への対応がこの分岐である。メモリを完全に隔離するサンドボックスではなく、サイズ制限とタイムアウトを併用する。

実装：`static.py/cli.py/_worker.py`、`check_apk/static_stage/configure_memory_limit`。

5.2 Manifestから読む情報

Androguardでバイナリ XMLを復元し、パッケージ名、版、minSdk、targetSdk、権限、Activity等のコンポーネントを保存する。復元Manifest自体も`evidence/AndroidManifest.xml`として索引化する。

権限は`uses-permission`等の宣言を読む。位置情報権限があるという結果は、「位置情報を取得した」「位置情報を送った」という結果ではない。SDKやライブラリのManifestマージで追加された権限について、最終APKだけから出所を確定する機能はない。

署名についてはv1/v2/v3の存在や証明書hash等のメタデータを保存する。暗号学的な署名の正当性検証は行わず、その状態を`not_performed`として出す。

5.3 通信セキュリティ設定

Manifestの`usesCleartextTraffic`、`networkSecurityConfig`参照、targetSdkによる既定値、NSCの継承関係とドメイン設定を読む。想定条件はAndroid API 24以上である。targetSdk 28以上では平文の既定値が禁止側になる等の条件を反映する。

複数のリソース候補から設定を一意に解決できない場合は、不明を保持する。信頼アンカーの宣言も保存するが、ドメイン、OS、ライブラリによる実動作を別に確認する必要がある。

実装：`static.py`、`analyze/network_config`。

5.4 DEX・リソースから抽出する情報

対象	抽出方法	所在として残す情報
複数DEX	`classes*.dex`内の文字列・クラス・命令を走査	DEX名、文字列index、クラス、メソッド、命令byte offset
API参照	`invoke-*`と登録パッケージ接頭辞を確認	呼出し先表現と命令位置
リソース・assets	UTF-8 / UTF-16LEでURLパターンを探索	ファイル名、デコード方式、文字offset
コンパイル済み文字列	`resources.arsc`の文字列プールを解決	リソース名、locale

URLパターンはHTTP(S)とWS(S)を対象にし、`url_candidate()`でホスト、ポート、schemeへ分解する。明示ポートがない場合はschemeから80/443を補う。元文字列と所在の組で重複を避け、所在を残した候補の一覧を作る。各リソースのバイト走査には8 MiBの制限がある。

5.5 SDK検出は特徴一致による候補提示

登録SDK	使う特徴の例
Firebase Analytics	`Lcom/google/firebase/analytics/`、Manifest文字列
Google Mobile Ads	`Lcom/google/android/gms/ads/`、広告設定のManifest文字列
Facebook App Events	`Lcom/facebook/appevents/`、アプリID設定
Sentry Android	`Lio/sentry/android/`、DSN設定文字列

クラス接頭辞だけの一致と、Manifest特徴も含む複数種類の一致を区別する。ルール版と参照元も保存する。SDKが初期化されたか、どの通信を所有するか、難読化後も同じ精度で検出できるかは、この処理からは分からない。

5.6 静的解析の完了状態

`coverage.processed`、`unprocessed`、`warnings`を保存する。親はDEX未処理、走査制限、一部のunresolved/partial警告等を見て`partial`を設定する。`completed`は実装した解析範囲内の処理状態であり、ネイティブコードや全経路を網羅したという意味ではない。

実装：`static.py/core.py/data/sdk_rules.json`、`analyze/url_candidate/rules`。

6　補助機能：URL引数の局所的な文脈解析

6.1　通常の文字列抽出との違い

research_context.pyは、文字列がどのAPIの引数に入るかを限定範囲で調べる補助機能である。通常のstatic.pyの抽出結果へ自動的に混ぜず、context.jsonとして分離する。研究用診断ランナーが明示的に呼ぶ。

対象は次の4シグネチャである。

- java.net.URLのString引数を取るコンストラクタ
- java.net.URIのString引数を取るコンストラクタ
- okhttp3.Request.Builder.url(String)
- WebView.loadUrl(String)

6.2　レジスタの値を追う方法

DEXの基本ブロックごとにレジスタ表を初期化し、const-string、move-object、move-result等を順に解釈する。定数文字列の連結と、限定したStringBuilder操作を扱う。IntentのgetDataString、getStringExtraは、値を確定せず実行時入力の由来タグを付ける。

たとえば定数"http://"と"example.invalid/feed"が同じブロック内の対応する連結に流れれば、URL引数候補を復元できる。一方、単に同じメソッドにURL文字列と通信APIが存在するだけでは、その引数へ渡ったとは扱わない。

status	内容
constant_url_argument	追跡範囲で定数URLがAPI引数に対応
constant_non_url_argument	定数は得られたがURL候補として解釈できない
runtime_value_argument	Intent由来等のタグを持つが値は実行時依存
unresolved_argument	追跡範囲では値を解決できない

未対応命令ではレジスタ・Builderの状態を保守的に無効化する。基本ブロック間、メソッド間、到達可能性、反射、ネイティブ、一般的なリソース参照は追わない。上限は既定で15万メソッド・200万命令、別ワーカーのタイムアウトは180秒である。

URL構築や要求設定の引数が分かることと、実際に送信したことは別である。

実装：research_context.py、analyze_block/scan_apk/scan_bounded。

7　動的取得：Androidをどのように動かすか

7.1　対象環境の確認とインストール

`Adb`クラスは明示した`emulator-5554`のようなシリアル形式の端末だけへ`adb -s`でコマンドを送る。実時間の締切と個別コマンドのタイムアウトを適用する。現行実装の対象はローカルEmulatorであり、任意の実機シリアルを受け付ける実装ではない。

自動取得は、端末状態、owner user、HTTP proxy、API/ABIの互換性を確認する。以前の問題となった`10.0.2.2:8082`のようなグローバルプロキシが設定されていれば、そのまま取得を始めず停止する。対照サーバーはプロキシとして使わない。

元APKと補助APKを`adb install -r`でインストールする。既存アプリデータは保持し、`pm clear`は行わない。インストール済みのAPKパスが1件であること、SHA-256が入力と一致すること、パッケージに対応するUIDが一意であることを確認する。

7.2　補助APKが必要な理由

PythonからActivityを起動できても、PCAPdroidが実際に開始したかは分からない。Javaの`ControlActivity`が公式CaptureCtrlを呼び、Androidの結果コードと状態を受け取り、補助APK自身のprivate領域へJSONを保存する。Pythonは`run-as`でそのファイルを読み出す。

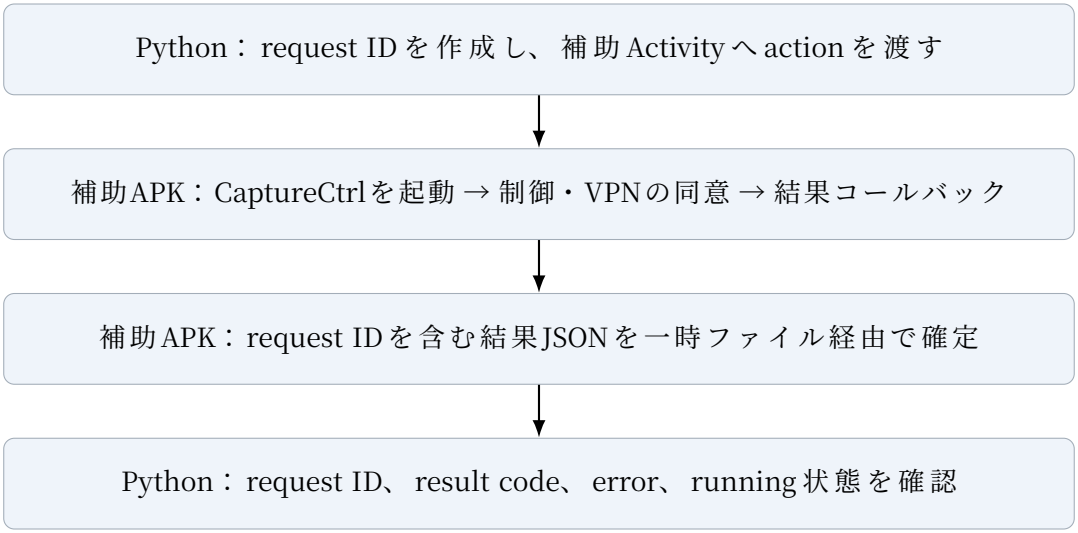


図 2: 制御応答の取得。Android側の同意は利用者が行う。

`Bridge.request()`は通常120秒までコールバックを待ち、request IDが一致し、`result_code==-1`でerrorがない場合を受理する。応答は`raw/bridge-results.json`へ記録する。既に他の取得が動いている場合は開始を拒否する。

実装：`android.py/automatic.py/ControlActivity.java`、`Adb/installed_uid/installed_hash/Bridge.request`。

7.3 PCAPdroidの取得設定

補助APKはPCAPdroid 2.0.2を確認し、PCAPファイル出力、UID拡張、全payload保存を要求する。非root、TLS復号なし、SOCKSなしで動かす。QUICを強制遮断する設定やPrivate DNSを自動遮断する設定は使わない。

「全payload保存」は、暗号化通信を平文にするという意味ではない。TLS通信では暗号化されたバイト列が保存される。表示側のプロファイルがmetadataでも、原本PCAPには平文HTTPの本文やヘッダー等が含まれ得る。

7.4 前後対照で取得経路を点検する

取得開始後、Macのloopbackに立てた`ControlServers`へ補助APKからHTTP要求を出す。Emulatorからは`10.0.2.2:<port>/health`（`<port>`は割当ポート）へ接続する。応答200と、同じrequest IDを含むサーバー受信記録を確認する。シナリオ実行後にも同様の要求を行う。

後段の`capture_gate()`は、制御応答、状態`False→True→False`、インストール同定、2件の端末／サーバー対照、補助UIDのHTTPフロー、cleanupを点検する。取得開始・停止と前後の経路に関する肯定的な根拠が得られるが、窓内の全通信の取得を保証するものではない。

7.5 登録された操作の実行

操作	実行方法	確認方法
`launch`	monkeyで対象のLauncherを起動	対象がforegroundになったか
`open_uri`	明示ActivityへVIEW Intentを送る	対象foreground、URI・Activityを記録
`tap`	resource IDのbounds中央をタップ	enabled、対象foreground、期待ID
`home`	HOME keyevent	対象foregroundからの移行
`wait`	指定した秒数待つ	全体締切に収まるか

各操作の開始・終了UTC、成功、phase等を`ui_events.jsonl`へ保存する。`open_uri`成功は配送とforegroundの確認であり、アプリ内部でのフィード解析成功を意味しない。許可画面以外の追加操作は、実験終了時の自己申告として別に記録し、客観的に全入力を監視した扱いにはしない。

実装：`automatic.py/research_diagnosis.py/ControlActivity.java、execute_steps/execute/capture_gate`。

7.6 停止・回収と失敗時の後始末

開始を確認して所有した取得は`finally`で停止し、停止状態を確認して`adbpull`する。後始末には別の時間予算を与える。開始要求を送ったが応答を得られない場合は所有が不明なため、別セッションを無条件に停止せず`manual_cleanup_required`を残す。補助スイートは対象アプリのforce-stopも試行後に行う。

8 原本を共通の通信フローへ変換する

8.1 アダプタという設計

アダプタ	入力	読取り上の特徴
`app-flow`	契約に沿うJSONL	UID、方式、観測段階等を厳密に検証
`pcap`	通常PCAP/対応PCAPNG	送信元アプリのUIDは得られない
`pcapdroid-csv`	2.0.2の固定列CSV	表示情報からHTTP/TLS成立を推定しない
`pcapdroid-pcap`	2.0.2 UID拡張PCAP	拡張の構造を確認しUIDを読む

`import_capture()`はmetadataのrun ID、APK hash、時刻、ツール版、前後対照等を検証する。入力サイズ上限は512 MiB。同じhashと同じadapterの再取込みは拒否する。異なる取得元はsource IDを分け、同一接続として自動統合・通信量合算はしない。

8.2 正規化フローのフィールド

フィールド	内容
`flow_id/source_id`	この出力内のフローIDと取得元ID
`destination_ip/port/transport`	宛先IP、宛先ポート、TCP/UDP
`host/host_evidence`	判明したホストと、HTTP Host / SNI等の由来
`uid/attribution`	取得器のUID情報と対象・別UID・不明等の区分
`observation`	接続試行、TCP成立、HTTP観測、取得失敗等
`application_protocol`	HTTP、TLS等。推定できない場合はnull
`started_utc/ended_utc`	補正した観測時刻
`related_ui_event_ids`	時計の仮定下で対応した操作
`source_record/evidence_ids`	元行・パケット番号と証拠参照

パケット数、フロー数、HTTP要求数、指摘数は異なる単位である。たとえば1フローにC02・C05・C06の3指摘が付いても、3回の通信を意味しない。

実装：`capture.py`、`validate_meta/import_capture/normalize_app_rows`。

8.3 PCAPのまとめ方と観測段階

client IP一覧で外向き／内向きを区別し、local IP・port、remote IP・port、transportの組でセッションを管理する。同じ組でも60秒以上のidleや異なる初期sequenceのSYNで別世代へ分ける。FINを使った完全な接続ライフサイクル追跡ではない。

TCPのSYN、対応するSYN/ACK、ACKが見えればtransport成立を記録する。外向きの単一TCPセグメントから完全に読めるHTTP要求にHostがあればHTTP観測とする。完全な単一セグメントのClientHelloからSNIが読めれば、ホスト由来をSNI、方式をTLSとする。

SNIでホストが分かっても、TLSハンドシェイク完了やHTTPSの本文は証明しない。

現行PCAP経路はSNIだけで`tls_established`を設定しない。UDP/443という番号だけでQUICやHTTPSとも判定しない。TCPストリーム再構成、IPフラグメント再構成は未実装であり、IPv4フラグメントは集計してスキップする。IPv6の拡張ヘッダー等を完全に解釈する実装でもない。

バイト数はIP層の長さを加算する。再送やヘッダーを含み、本文サイズや利用者データ量とは一致しない。IPからドメインを推測するDNS対応付けは、PCAP経路では行わない。

8.4 PCAPdroid UID拡張の検査

2.0.2の拡張形式について、先頭の疑似Ethernet、IP長、4 byte境界のpadding、32 byte trailer、magic値`0x01072021`を確認する。App表示名はパッケージの同定に使わず、非負UIDだけを取り出す。

フロー内の**全パケットで有効UIDが一致**するときだけ、フローのUIDを採用する。欠落または不一致を含めば不明にする。形式検査は収集器が正しいUIDを付けたことの独立した証明ではない。

8.5 CSVは何を失うか

CSVでは既定列名を厳密に確認し、UIDの負値を不明にする。`Info`や翻訳済みStatus/Proto表示からSNI・HTTP要求・TLS成立へ昇格しない。`host=null`、`application_protocol=null`、`connection_attempt`として取り込む。列数が曖昧なら推測せず拒否する。

実装：`capture.py/pcapdroid.py`、`pcap_flows/_finish_pcap/trailer_uid/pcapdroid_flows`。

8.6 UIDとパッケージによる帰属

入力条件	区分
このrunのインストールUIDがない、または記録UIDがない	`unknown`
UID一致、パッケージ記録なしまたは対象と一致	`target`
UID一致だがパッケージ名が別	`unknown`（共有／矛盾）
UID不一致だがパッケージ名だけ対象と一致	`unknown`
別UIDで収集器が委譲等の関連候補を注記	`related_candidate`
それ以外の別UID	`other`

宛先が研究サーバーだからという理由や、アプリ名が似ているという理由で、不明を対象へ変換しない。shared UIDやOS等への委譲があるため、UID一致にも解釈の制約がある。PCAPdroid画面の`Unknown(-1)`は、このデータモデルでは「対象の通信なし」へ置き換わらない。

8.7 取得品質がpartialになる理由

`import_capture()`は、前後対照passed、欠落区間なし、coverageの申告、時刻補正状態、fragmentスキップ・UID不明パケットの有無、capture failedの有無などから品質を設定する。

自動取得経路は`coverage_status=unknown`、`clock_alignment_status=conditional`を明示するため、通常は`partial`となる。これは実行操作の失敗や、取得済みHTTP要求の無効化と同義ではない。条件を満たす事実は個別に使い、完全性の不足も同時に残す。

旧metadataでは一部の状態項目が省略されると、取込み側の既定値が使われる。このため`completed`というラベルだけで比較せず、元metadataと取得経路を合わせて読む。過去結果の検証スクリプトは、生PCAP・停止統計・操作原本等も別途点検する。

通信失敗はアプリの接続結果、取得失敗は観測系の状態である。

対象アプリのHTTPタイムアウトが出ても、接続試行が収集できれば照合可能なことがある。収集器が止まっていた場合には、それを「要求が出なかった」という観測にしない。

実装：`capture.py/automatic.py`、`attribution/import_capture/execute`。

9 時計補正と操作区間の対応

9.1 PCとAndroidの時刻は同じとは限らない

PCで取得した時刻とAndroidのパケット時刻に差があるため、次の方向で補正する。

$$t_{\text{host}} = t_{\text{device}} + \Delta. \tag{1}$$

`clock_sample()`は、PC側の時刻aを取り、ADBで端末時刻dを読み、PC側の時刻bを取る。端末時刻の表示精度qも考え、補正値の候補区間を

$$\Delta \in [a - d - q,\ b - d] \tag{2}$$

とする。実時間とmonotonicの経過差も確認し、測定中の大きな時計不連続を拒否する。

開始前と停止後の2測定の共通部分を$[L, U]$とし、共通部分がなければ補正済みデータへ取り込まず原本を残す。代表値と許容幅は

$$\widehat{\Delta} = (L + U)/2, \qquad \varepsilon = (U - L)/2 \tag{3}$$

である。途中のクロックドリフトの上限までは測定していないため、状態は`conditional`とする。

9.2 操作への対応には2種類の条件がある

用途	判定に使う時刻	条件
一般の操作・Home対応	フロー開始時刻t_sの区間	$[t_s - \varepsilon, t_s + \varepsilon] \subseteq [a_e, b_e]$
登録入力先の許可例外	フロー開始・終了の全体	$[t_s - \varepsilon, t_f + \varepsilon] \subseteq [a_e, b_e]$

一般の対応では、誤差区間が完全に入ると操作IDを付け、境界に重なるだけなら`boundary_uncertain`とする。Homeの切替は`phase_confirmed_utc`を開始境界として使う。C06が扱うのは主に「Home後に開始したフロー」であり、Home前から続く接続の全送受信をパケット単位で分割する方式ではない。

入力先の例外は、フロー全体が成功した入力操作区間に収まることを要求する。両者を同じ条件として説明すると、実装より広い許可を意味してしまう。

実装：`automatic.py/capture.py/intent_extensions.py`、`clock_sample/import_capture/recorded_input_verdict`。

9.3 境界の具体例

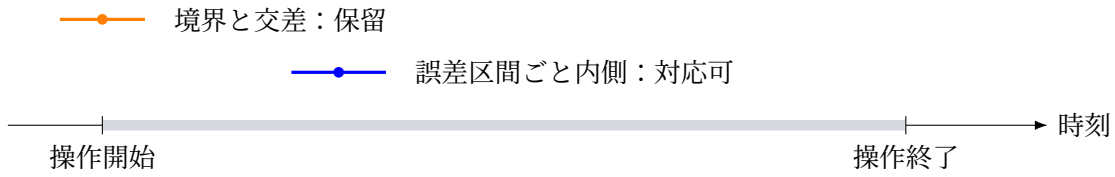


図 3: 操作対応は点推定だけで決めず、補正の幅を使う。

説明用に操作区間を10.000〜40.000秒、$\varepsilon = 0.013$秒とする。フロー開始が10.005秒なら誤差区間は9.992〜10.018秒で、開始境界にまたがる。代表時刻が区間内でも、確実な対応としては扱わない。開始が12秒・終了が13秒なら、誤差を広げても全体が内側になり、他の条件も満たせば入力例外を評価できる。

この数値例は計算の説明であり、新しい端末測定ではない。過去の時計測定で得た約0.26〜0.29秒の補正区間を、すべての試行へ固定して使う実装でもない。自動経路は各試行の前後測定を使う。

9.4 C06に影響する点

`background_network_policy=forbid`でも、操作区間が判明しなければC06は判定不能になる。対応区間が判明し、`after_home=true`であれば不一致候補を出す。foregroundやHome後という操作状態は、Androidのすべてのプロセス状態やサービス実行状態の分類とは区別する。

9.5 入力許可に影響する点

ホスト・ポートが登録URIと一致しても、時刻許容幅が不明、フローが操作終了を越える、操作記録のhashが合わない等の場合は、入力先だから許可とせず判定不能とする。URIの許可はネットワーク全体の恒久的な例外ではなく、記録された操作と観測の組に対する条件付きの評価である。

10 開発意図との照合器

10.1 出力モデル：4判定と独立した観測情報

verdict	読み方
consistent_with_observed_evidence	今回比較できた条件と、観測した証拠の範囲で整合
mismatch_candidate	明示した条件と反する宣言・観測の候補
needs_review	意図が未指定、静的文字列等で手元の確認が必要
not_evaluable	帰属、時刻、方式、観測範囲等が不足し比較できない

指摘にはfinding_id、rule_id、subject、verdictだけでなく、observation、attribution、content_visibility、evidence_ids等を保存する。静的な権限宣言の判定と、対象UIDの通信要求の判定を同じ種類の観測として扱わない。

10.2 規則一覧

ID	対象	比較する条件	主な不足時の扱い
C01	対象の接続	通信禁止	非対象・帰属不明はC07へ
C02	対象の宛先	許可ホスト・ポート・方式、登録入力	ホストや方式不足は判定不能
C03	SDK候補	許可SDK ID	未指定なら要確認。未検出を不存在にしない
C04	権限宣言	許可権限名	未指定なら要確認。使用の有無は別
C05	設定／観測	平文通信の条件	設定未解決、方式不明は判定不能等
C06	操作区間	Home後の通信条件	操作時刻との対応不足は判定不能
C07	候補と不足	静的URL、帰属、取得・解析範囲	理由ごとに要確認／判定不能

10.3 評価順序

静的な宣言・候補を評価した後、フローごとに取得失敗を先に除外し、次にattribution!=targetをC07として扱う。その後に対象の宛先条件を比較し、さらにC05とC06を別々に評価する。したがって、帰属不明を通信禁止の違反へ変換しない。

単一フローに複数の規則が当たる場合、1つの優先度で他の指摘を消去しない。たとえばC02不一致候補、C05整合、C06判定不能という3行が同時に成立する。規則ごとの問いが違うからである。

実装：compare.py、compare/list_verdict/destination_verdict。

10.4 通常の宛先許可リスト判定

`destination_verdict()`は次の順に評価する。

1. 未指定なら要確認、通信禁止なら対象の試行を不一致候補とする。
2. ホストがない場合は、明示IPの許可先で直接比較できるか確認する。ドメインとIPを勝手に対応付けない。
3. ホストとサブドメイン条件に合う候補を集める。
4. 合うホストで許可ポートがなければ不一致候補とする。
5. 方式の制約が空ならホスト・ポートの整合として返す。
6. 方式が指定されているのに観測できなければ判定不能とする。
7. HTTP等が一致すれば整合、不一致なら不一致候補とする。ただしTLSだけではHTTPSと反する証拠にならないため、HTTPS許可へのTLS観測は保留する。

意図	観測	結果の例
`api.example.invalid:443`でHTTPS	宛先IPだけ	ドメインを確定できず判定不能
同上	同ホスト、443、TLSのSNI	HTTPSのアプリ意味は不足し判定不能
同ホスト、443、方式制約なし	同ホストとポートが判明	その宛先条件の範囲で整合
明示IPの18762だけ許可	同IPの18761へHTTP	ポート条件の不一致候補

10.5 C05・C06の並列評価例

意図が「入力先のみ、平文許可、Home後禁止」で、入力と別ポートへforeground中のHTTP要求があれば、C02は不一致候補、C05は平文許可と整合、C06は対応区間がforegroundなら整合になる。Home時刻の対応が分からなければC06だけ判定不能になる。

C05の設定評価は`network_config`の解決結果を使い、実通信のC05は観測方式を使う。`usesCleartextTraffic`が許可されているだけで「平文要求を観測した」とはしない。C05の観測側は平文禁止の場合にHTTP/DNSを平文として扱い、未分類通信を自動的に暗号化済みにしない。

10.6 登録入力先を許可する1.1拡張

`recorded_input_verdict()`は、単に「動的のみだから入力先だろう」と説明する代わりに、次の証拠条件を評価する。

1. 明示固定許可先で整合する場合は、その根拠で返す。
2. 非負の対象UIDがrunにあり、フローUIDが一致し、TCPであることを求める。
3. シナリオとUIイベントが索引に登録され、hashが一致することを確認する。
4. 保存シナリオとrun内の登録内容が同一であることを確認する。
5. 成功した`open_uri`イベントのID・URI・Activityが登録ステップと一致するものだけを採用する。
6. 宛先ホスト・ポート、許可schemeを照合する。
7. 時計許容幅を含むフロー全体が、その入力イベントの区間内に収まることを求める。
8. 観測アプリ方式が入力schemeの条件を満たすか、通常の宛先判定を使って比較する。

$$\begin{aligned}\mathrm{InputAllowed}(f) \Leftarrow\ & \mathrm{UIDMatch}(f) \wedge \mathrm{TCP}(f) \wedge \mathrm{IntactRecords} \\ & \wedge \mathrm{RegisteredSuccess}(e) \wedge \mathrm{EndpointMatch}(f, e) \\ & \wedge \mathrm{IntervalContained}(f, e) \wedge \mathrm{ProtocolMatch}(f, e).\end{aligned} \tag{4}$$

上式は説明用に条件を要約したもので、固定許可先の先行評価等は列挙手順に従う。

結果には`permission_basis`を付ける。入力条件に整合した場合は`recorded_input_endpoint`、入力先と固定許可先の外なら`outside_registered_and_explicit_endpoints`、操作記録不足や時刻不確実ならそれぞれの理由を残す。

10.7 この例外が許可しないもの

入力されたURIのホスト・ポート・schemeを評価する方式であり、URLパスごとのアクセス制御、リダイレクト先への自動継承、DNS別名の自動許可、任意の手入力の自動記録は行わない。入力許可先へ秘密情報を送る場合でも、宛先条件だけは整合し得る。本文と利用目的は独立に確認する必要がある。

また、入力先の**外**であることが十分に確認できれば、入力区間との対応を評価する前に不一致候補を返す場合がある。端点が同じで例外を許す場合の条件と、端点が外であると判断する条件を分ける。

実装：`intent_extensions.py`、`indexed_file/recorded_input_verdict`。

11 静的・動的の対応表と日本語説明

11.1 対応規則はホスト／明示IPとポート単位

静的URL候補をホスト・ポート単位に集約する。既知のXML名前空間URL等は`reference_hints`として別枠へ移すが、`static.json`の元候補は保持する。この表示上の整理は、`compare.py`の全静的URL指摘を削除するフィルタとは別である。

対象UIDのTCPフローについて、ポートとホスト、または明示IPが一致すれば`both`にする。URLパスやアプリ方式はこの対応規則の一致キーではない。政策上の許可／禁止と混同しない。

コード 3: 実装抜粋：対応表の一致規則。

```
def corresponds(candidate, flow):
    if flow.get("observation") == "capture_failed" or flow.get("transport") != "TCP" or candidate["
    port"] != flow.get("port"):
        return False
    return (candidate["host"] == flow.get("destination_ip") if is_ip(candidate["host"])
            else candidate["host"] == flow.get("host"))
```

分類	条件の概要
`both`	静的端点候補に、規則で一致する対象フローがある
`static_only`	利用可能な取得があり、対応候補もない範囲で実測一致がない
`dynamic_only`	対象帰属・ホストがあり、静的処理completedで、一致候補がない
`indeterminate`	取得なし、候補となる帰属不明、ホスト不足、静的処理不足等

11.2 説明は事実・不足・仮説を別々にする

`explanation.py`は、対応表とrun・flows・staticを読み、各行の事実に出典フィールドと証拠IDを付ける。`evidence_gaps`には取得不足、静的不足、帰属不明、ホスト不明、方式範囲外等を別のcodeで記録する。実行時入力や未使用文字列の可能性は`unconfirmed_hypotheses`へ置く。

「動的のみ」は不正通信の同義語ではない。許可された入力先への要求も、別の固定先への要求も動的のみになり得る。この違いを判断するのが前節の意図照合である。説明層は照合結果を変更せず、コード上の因果原因も生成しない。

実装：`differential.py/explanation.py`、`corresponds/build/explain`。

12　証拠の保存とレポート生成

12.1　出力フォルダの構造

コード 4: 典型的なlive出力。ファイルはモードや失敗段階で増減する。

```
run/
  run.json
  intent.json
  scenario.json
  static.json
  flows.jsonl
  ui_events.jsonl
  findings.json / findings.csv
  correspondence.json / correspondence.csv
  difference-explanation.json / difference-explanation.csv
  summary.html / report.html
  automatic.metadata.json
  raw/
    original.apk
    automatic.pcap
    capture-1.pcap
    capture-1.metadata.input.json
    intent-declaration.json
    installation-identity.json
    analysis-code-manifest.json
    bridge-results.json
    automatic-controls.jsonl
    clock-samples.json
    static-worker.log
  evidence/
    index.json
    AndroidManifest.xml
    network_security_config.xml
```

raw/automatic.pcapは回収元、raw/capture-1.pcapは取込み後の取得元別コピーである。別の実験2件として数えない。findings.jsonは照合の構造化結果、correspondence.jsonは対応関係の構造化結果として読み分ける。

12.2　証拠IDから原本へ戻る

索引の各要素は、相対path、SHA-256、locatorを持つ。locatorはCSV行、PCAPパケット番号、Manifest、静的所在等を表す。指摘は証拠IDだけを参照し、索引を経由して原本位置へ戻る。

コード 5: 形式を説明する模式例。hash値は実データではない。

```
{
  "capture-1-flow-4": {
    "path": "raw/capture-1.pcap",
    "sha256": "<64 hexadecimal characters>",
    "locator": {"packet_numbers": [15,16,17,18,19,20,21,22,23,24]}
  }
}
```

実装：core.py/capture.py、add_evidence/import_capture。

12.3 HTML生成前に何を検査するか

`report()`は、指摘で参照する証拠IDが索引にあること、解決したpathが出力フォルダ内であること、ファイルが存在しhashが一致することを確認する。対応表を作る`build()`も、その行で参照する証拠を検査する。

検査対象は各出力で参照したファイルであり、フォルダ内の全ファイルを常に独立検証する方式ではない。hashと索引を同時に改変できる主体に対する署名保証でもない。保全の目的は、保存時の参照をたどり、欠落・変更を発見できるようにすることである。

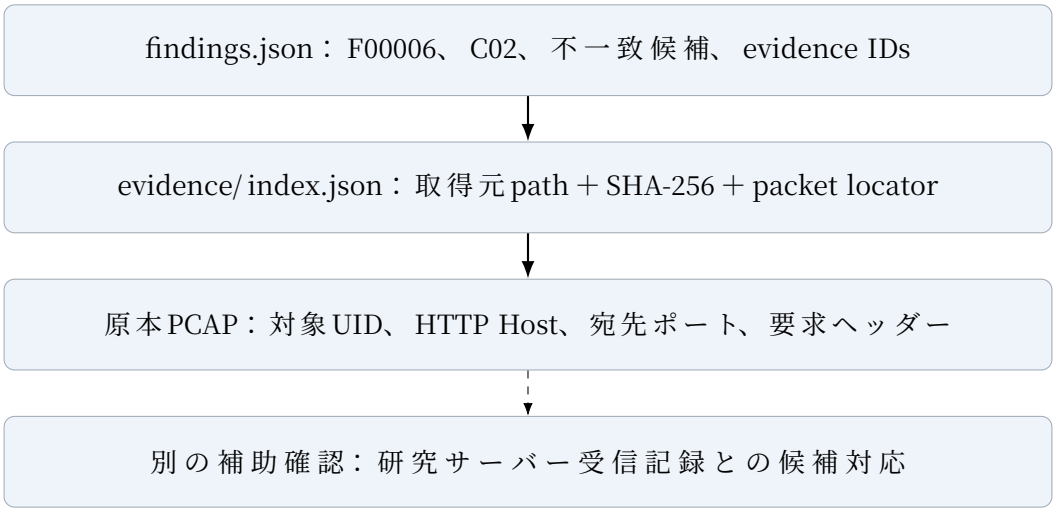


図 4: 照合の根拠と、追加のサーバー候補対応は区別する。

12.4 表示時の扱い

HTMLへ値を入れる前にescapeし、任意スクリプトを動かす用途にしない。CSVでは先頭が`=`、`+`、`-`、`@`等の値を無害化する。日本語CSVはUTF-8 BOM付きで保存する。

意図1.1の要約画面は、不一致候補を先に表示し、主な接続と不足等を抜き出す。最大30行の要約表示に対して、全指摘は`report.html`と`findings.json`に残る。件数は指摘数であることを明記する。

12.5 共有用と原本付き資料

通常のrunは元APK、平文本文、端末情報などを含み得る研究者用資料である。`share`は最小限の要約を別出力へ作る機能で、原本一式を匿名化したものと同義ではない。外部共有時は公開範囲を確認する。この技術PDF・ソースZIPには生PCAPや元APKを含めない。

実装：`report.py/differential.py/intent_overview.py/share.py`、`report/build/section/export_summary`。

13　実際の1試行を処理順に読む

13.1　fixed-001の入力と観測

主評価intent-seeds-001のfixed-001を例にする。意図は登録入力先のみ許可で、入力は10.0.2.2:18762だった。一方、素材は別の固定先10.0.2.2:18761へHTTP要求を出す仕様である。

段階	保存された内容と読み方
意図	固定の許可先は空、登録したHTTP(S)入力先を許可
静的結果	URL文字列候補0。補助メソッドによる構築という自作素材の条件
操作	18762を含むURIを対象Activityへ配送し、操作区間を記録
取得	18761宛の対象UIDフロー、HTTP要求の記録
正規化	capture-1-flow-4、パケット15〜24、GETヘッダー18番
照合	F00006/C02/mismatch_candidate
根拠の種類	outside_registered_and_explicit_endpoints

登録された端点は18762、観測した端点は18761なので、登録入力の許可条件と合わない。フローは対象帰属として扱えるため、帰属不明の保留ではなく宛先条件の不一致候補となる。静的候補0は、自作文字列抽出器と素材構成の組合せにおける結果である。

13.2　差分と意図照合が違う出力を持つ理由

同じ主評価のinput素材も、URL文字列候補0で、入力先へのHTTP要求を観測した。したがって両素材とも静的・動的の差分では「動的のみ」になり得る。意図照合では、入力先に整合した要求と、入力外の固定先を分けられる。

素材	静的候補	動的	意図との主要照合
input	URL候補0	登録入力先へHTTP	整合
fixed	URL候補0	別ポートへHTTP	不一致候補
delayed	URL候補0	30秒窓で対象未観測	判定不能を保持

13.3　サーバー記録はどう使うか

研究用サーバーとの候補対応は、端点、試行tokenやnonce path、対象UID、補正時刻等を別スクリプトで確認する。C02の一般入力にサーバー記録を自動統合して判定するわけではない。NATで端末の送信元portとサーバーのpeer portが違うため、5-tuple完全一致の接続同一性を証明した扱いにもしていない。

実装：compare.py/intent_extensions.py/docs/research/intent-seeds-001-2026-11-06、findings/trial-results/review。

14　研究用ランナーの仕組みと実験の位置付け

14.1　意図照合スイートは既存パイプラインを再利用する

`intent_seeds.run_suite()`は、input・fixed・delayedの3素材を各3回、計9試行の計画として作成する。入力のarmは反復ごとにA/B/A、実行順はinput→fixed→delayedを繰り返す。通常解析を素材別の新規出力へ呼び、取得対照や宣言時点の条件が崩れた場合は後続を停止する。

開始前に素材APKのinventory hash、意図、計画、期待する接続結果、コード一覧を保存する。終了時は追加操作の自己申告、サーバー記録、証拠hash一覧を残し、返却ZIPを作る。作者が素材・期待値・解析器を設計した条件なので、目的は規則と取得を接続した動作確認である。

確認済み主評価	input 3試行	fixed 3試行	delayed 3試行
対象HTTP要求	各1件	各1件	30秒窓では0件
主要出力	整合3/3	不一致候補3/3	判定不能3/3
取得品質	partial	partial	partial

delayedは75秒後に送る仕様で、30秒観測の後に停止する。ここで確認したのは遅延通信を検出したことではなく、未観測を無通信へ置き換えず保留する規則動作である。9試行で帰属不明フローは0であり、帰属不明を保持する実例は補助群の過去UDPデータにある。

14.2　入力切替診断は任意の補助経路

`research_contrast.py`は、公開アプリ等の登録URIをA/Bへ切り替え、候補要求とサーバー受信を評価する。`research_diagnosis.select_probe()`は、宣言した仮説ペアを分離できる数を実行コストで割った規則で候補操作を選ぶ。既定候補は入力切替と同じ入力の反復であり、汎用的な機械学習や最適な因果探索ではない。

補助結果ではA/B/B/Aの4試行で入力ポートへの追随を観測した。これは対象アプリ・登録操作の下での関連の根拠であり、一般のコード上の根本原因や漏えいを同定する機能とは区別する。

14.3　実装済み機能と実行済み評価を分ける

コードには追加比較用の4素材20試行計画等もあるが、その存在を今回の実測件数に加えない。主評価9、従来の補助29、3モード再解析による27出力は別の単位である。動的のみと併用の主通信判定は今回の再解析で同じであり、併用による一般精度向上を示した結果ではない。

実装：`intent_seeds.py/research_contrast.py/research_diagnosis.py`、`run_suite/run_live/select_probe`。

15 実際に使うときの読み方と操作

15.1 一般の統一CLI

以下はコマンドの説明例であり、この資料作成時に新しい端末実験は行っていない。プロジェクトの対応版と、対象のシナリオが用意されている場合に使う。

コード 6: プロジェクトルートからのlive解析例。出力先は新規にする。

```
source .venv/bin/activate
apk-audit doctor --serial emulator-5554
apk-audit analyze \
  --apk apks/control-http.apk \
  --serial emulator-5554 \
  --intent examples/intent-recorded-input-v1.1.json \
  --scenario examples/scenario-http-button.json \
  --out runs/mechanism-example-001
open runs/mechanism-example-001/summary.html
```

上の`scenario-http-button.json`はボタン操作の例であり、`open_uri`による登録入力先を証明するシナリオではない。そのため登録入力例外が必要な評価には、その対象ActivityとURIを持つ別シナリオを用意する。

15.2 配布拡張ランナーを使っている場合

`apk-audit-intent-v1/run_intent.py`は、bundleと既存プロジェクトの互換coreのhashを確認する。既存`src`を書き換えず、Python package pathへ`payload`を追加して拡張を優先する。対応版が違う場合は上書きせず停止する。

コード 7: 配布版の個別解析の形。各パスは実際の配置に合わせる。

```
"$HOME/Documents/apk-audit-unified-v1/.venv/bin/python" \
  "$HOME/Documents/apk-audit-intent-v1/run_intent.py" \
  --apk "$HOME/Downloads/target.apk" \
  --intent /path/to/intent.json \
  --scenario /path/to/scenario.json \
  --out "$HOME/Documents/apk-audit-unified-v1/runs/intent-example-001"
```

本資料が説明するのはクラウド側で確認した現行ソースであり、利用者のMac内の全コピーが自動更新されたことを意味しない。どの版が動いたかはrunの`analysis-code-manifest.json`と配布manifestを確認する。

15.3 動的取得を行わない使い方

`--static-only`は動的証拠を意図的に除外したことをC07へ残す。既存取得を再解析する場合は`--capture`と`--metadata`、対応する`--adapter`を指定する。再解析は新しい端末試行ではなく、過去の取得原本へ別の処理を適用したものとする。

15.4 レポートを読む順序

1. summary.htmlで対象APK、実行モード、段階別状態と主な意図照合を確認する。
2. 不一致候補のsubjectを見て、権限宣言、SDK候補、対象の接続等を区別する。
3. verdictに加え、観測段階・帰属・方式・permission_basisを読む。
4. 証拠IDからreport.htmlの索引へ進み、原本pathとlocatorを確認する。
5. 対応表と差分説明で、静的候補があるか、何の情報が不足しているかを調べる。
6. run.jsonとmetadataでシナリオ、初期データ、時刻仮定、errorsを確認する。

15.5 よくある状況と確認箇所

状況	主な確認箇所	読み方・対応
Emulatorが見つからない	adbdevices-l、runのerrors	選択したEmulatorを起動し、新しい出力名で実行
No Internet	Androidのproxy/VPNとサーバー状態	対照サーバーをHTTPプロキシに設定しない
bridge-results.jsonがない	先にrun.jsonの元エラー	端末未起動等でbridge要求前に止まった可能性
HTTPアプリがタイムアウト	アプリログ、PCAP、サーバー	通信失敗と取得失敗を分ける
Unknown UID	PCAP拡張／CSV、インストール同定	宛先だけで対象へ割り当てない
partialだが対照合格	metadata、clock、coverage	観測済み事実と完全性不足を併記
出力名が既に存在	runsの保存済み結果	保持して新規名を使う

15.6 手作業が必要な部分

初回のAndroid Studio／AVD設定、制御・VPN同意、対象アプリの適切なシナリオ設計、意図JSONの作成、アプリ固有画面の意味の確認は利用者が行う。公開アプリの購読や再生など、登録していない操作を加えた場合は実験条件に記録する。独立評価者の確認をAIが実施したことにはしない。

15.7 この資料の作成で行った検査

ソースをモジュール・関数単位で照合し、説明用JSONを既存のvalidatorで検証した。PDFはXeLaTeXで生成し、相互参照、文字切れ、図表、ソース同梱物を確認した。これは技術資料の検査であり、Android取得の新しい実測や一般精度の再評価ではない。

16 実装の境界と改善する場合の場所

改善したい点	現実装	変更・評価する場所
URLの解決範囲	文字列抽出＋別経路の基本ブロック内文脈	`static.py/research_context.py`。未知操作・難読化・nativeを含む独立素材で検証
SDKの種類と強さ	4登録ルール、接頭辞とManifest文字列	`sdk_rules.json`。実SDK、難読化、誤検出の評価を別に設計
通信の読取り	単一セグメントHTTP/SNI、フロー集約	`capture.py`。TCP再構成・再送・順序ずれ・分割を扱うテストが必要
QUIC / TLS本文	ポートだけで方式判定せず、復号しない	新adapter等。端点と内容の観測範囲を分けて評価
取得元の自動突合	別source保持。研究用候補対応は補助	5-tuple、NAT、nonce、時刻誤差を含む候補モデルと誤結合評価
Home後の送受信	フロー開始時刻を操作区間に対応	パケット単位の時間切分けを導入し、既存のC06との意味差を検証
操作の網羅	登録したlaunch/tap/open URI等	シナリオ追加か別の探索器との連携。状態・失敗・再現性も評価
意図の表現	宛先、権限、SDK、平文、Home後	型と規則を追加。自然言語の目的を自動的に正解へ変換しない
独立評価	主評価は自作素材の動作確認	原本付き確認票・第三者確認・追加アプリ等を研究目的に合わせ選択

16.1 現在の研究と実装が示すこと

本実装は、開発者の構造化意図を中心に、静的宣言・候補と動的観測を照合し、その判定に戻れる証拠参照を作る。自作3条件各3回では、設計した規則が期待した整合・不一致候補・保留を出すことを確認した。

静的と動的を併用する理由は、異なる証拠を並べて読み、宣言だけの候補や実行時に現れる宛先を別々に点検できることである。今回の主通信判定で動的単独を上回る一般精度を実証した、という結論ではない。プライバシーの自己確認を支援する位置付けであり、安全性や漏えい不存在の証明は出力しない。

A　ソースコードへの案内

以下の表は2026年11月9日時点の関数定義位置から自動生成した。行番号は配布パッチ等で変わるため、SOURCE-MAP.csvとCODE-MANIFEST.jsonも同梱する。hashはこの説明がどのソースを参照したかを識別するための値で、実行の独立した証明ではない。

モジュール（src/apk_audit/）	関数・クラス	定義行
cli.py	main	173
cli.py	static_stage	14
pipeline.py	scenario_input	12
pipeline.py	analyze	34
core.py	normalize_host	92
core.py	new_run	138
core.py	add_evidence	159
contracts.py	validate_intent	43
contracts.py	validate_open_uri	72
_worker.py	configure_memory_limit	10
static.py	check_apk	19
static.py	network_config	49
static.py	analyze	108
research_context.py	analyze_block	52
research_context.py	scan_bounded	202
android.py	Adb	16
automatic.py	installed_uid	25
automatic.py	installed_hash	34
automatic.py	Bridge	44
automatic.py	clock_sample	77
automatic.py	execute_steps	96
automatic.py	execute	151
capture.py	validate_meta	28
capture.py	attribution	62
capture.py	pcap_flows	163
capture.py	_finish_pcap	256
capture.py	import_capture	276
pcapdroid.py	trailer_uid	20
pcapdroid.py	pcapdroid_flows	61
compare.py	destination_verdict	21
compare.py	compare	55
intent_extensions.py	indexed_file	40
intent_extensions.py	recorded_input_verdict	50
differential.py	corresponds	22
differential.py	build	37
explanation.py	explain	30
report.py	report	21
intent_overview.py	section	6
intent_seeds.py	run_suite	16
research_diagnosis.py	select_probe	15
research_diagnosis.py	capture_gate	132
research_diagnosis.py	request_witness	167
research_contrast.py	run_live	98
share.py	export_summary	9

B　実装抜粋：帰属と入力区間

B.1　正規化後の帰属

以下はcapture.pyからの抜粋である。PCAPdroidの負UIDはadapter段階で不明へ正規化され、この関数はrunのインストールUIDと比較する。

コード 8: 帰属区分の実装。

```
def attribution(row, run):
    package, uid = row.get("package"), row.get("uid")
    expected_package, expected_uid = run.get("package"), run.get("target_uid")
    # UID is specific to this installation. Conflicting or incomplete identity stays unknown.
    if not isinstance(expected_uid, int) or not isinstance(uid, int):
        return "unknown", "installation UID or record UID missing"
    if uid == expected_uid:
        if package and package != expected_package:
            return "unknown", "shared/conflicting UID: package differs"
        return "target", "UID matched this run; shared UID attribution remains a limitation"
    if package == expected_package:
        return "unknown", "package matched but installation UID differs"
    if row.get("related_to_target") is True:
        return "related_candidate", "collector annotated possible delegation; not proven"
    return "other", "record UID differs from target installation UID; delegation not excluded"
```

B.2　入力許可の時刻条件

以下はintent_extensions.pyの一部である。補正後の開始・終了を誤差分だけ広げ、入力イベントへ完全に含まれることを求める。周辺のUID、操作同一性、端点、方式検査も満たす必要がある。

コード 9: 登録入力例外の区間条件の抜粋。

```
epsilon=flow.get("clock_alignment_half_width_s")
if type(epsilon) not in (int,float) or not math.isfinite(epsilon) or epsilon<0:
    return UNKNOWN,"Clock uncertainty for the input interval is unavailable",basis,"clock_unknown"
try:
    start,end=timestamp(flow["started_utc"]),timestamp(flow["ended_utc"])
    active=[r for r in matching if timestamp(r[0]["started_utc"])<=start-epsilon
            and end+epsilon<=timestamp(r[0]["ended_utc"])]
except (KeyError,AuditError):active=[]
if not active:return UNKNOWN,"Endpoint matches an input but interval correspondence is uncertain",
    basis,"input_interval_uncertain"
```

C　用語と実装資料

用語	この資料での意味
APK	Androidアプリの配布ファイル。ここでは単体APK
Manifest	パッケージ、権限、Activity等の宣言
DEX	Androidで動くコードを収める形式。Pythonソースではない
UID	そのAndroidインストールにおけるユーザーID。接続元帰属の根拠
PCAP	パケット原本の保存形式。通常の形式だけではアプリ名を含まない
VPN方式	PCAPdroidがAndroidのVPN APIでトラフィックを扱う方式。root取得とは別
HTTP Host	HTTP要求内の接続先ホスト情報。本文や目的とは別
TLS SNI	ClientHello内のサーバー名情報。見えない場合もある
フロー	パケットを端点・方式等で集約した記録。必ずしもHTTP要求1回ではない
正規化	取得元ごとの差を共通のフィールドへ変換する処理
照合	明示した意図の条件と証拠を規則で比較する処理
候補対応	条件に合う観測・受信記録の対応。完全な接続同一性の証明とは別
証拠索引	IDから原本path、hash、元位置へ戻る辞書
再解析	保存済み取得を別の処理で読むこと。新しい端末実験ではない

外部ライブラリや収集器の仕様は、各公式資料に従う。以下の一覧は実装理解の入口であり、本文未確認の研究を読了として扱うものではない（確認日：2026年11月9日）。

- Androguard：`https://androguard.readthedocs.io/`
- dpkt：`https://dpkt.readthedocs.io/`
- JSON Schema Draft 2020-12：`https://json-schema.org/draft/2020-12`
- Android Debug Bridge：`https://developer.android.com/tools/adb`
- Android Network Security Configuration：`https://developer.android.com/privacy-and-security/security-config`
- PCAPdroid：`https://github.com/emanuele-f/PCAPdroid`

PCAPdroid 2.0.2のCSV／UID拡張に関する実装参照版は`66f7b3514540c20add8655a74dfee93d00a9563b`。本資料の主たる根拠は付録Aのローカルソースと付属hash一覧である。